M365 Name Pronunciation.

Summary of the feature

Design and implementation you drove

Progressive rollout architecture: ringed/flighted rollout across Teams, Outlook, LPC surfaces, then special clouds (GCC/GCC High), with admin controls and staged exposure.

Backend migration architecture: PAPI → IAPI cutover through Substrate while preserving client behavior, updating contract shape, and reducing legacy assumptions.

Reliability architecture: incident-driven stabilization loop (IcM ownership, mitigation, closure), plus telemetry/scorecards to measure production health and adoption.

Surface expansion architecture: moved beyond host-specific integrations into broader Vanilla LPC consumers (including Copilot-related surfaces).

Technology choices

- Frontend/host integration: Teams + Outlook + LPC framework.
- Service/API path: Loki client, PAPI (legacy), IAPI over Substrate ImageB2.
- Storage: SDS for pronunciation item/audio blob retrieval, with mailbox-backed recording model.
- Ops/observability: telemetry pipelines, reliability dashboards, and IcM workflows for LiveSite response.
- Rollout controls: feature flights/rings, admin toggles, sovereign cloud enablement paths.

Key challenges faced (and handled)

- Migration risk: contract differences between PAPI and IAPI (payload shape/fields) required coordinated client-service alignment.
- Reliability regressions during rollout: missing pronunciation/404-style failures required rapid diagnosis and staged recovery.

- Tenant/environment misconfiguration issues: client-path/config mismatches impacted behavior until corrected.
- Cross-surface UI consistency: introducing pronunciation UI caused layout edge cases (for example header button shifting) that needed follow-up fixes.
- Compliance + cloud complexity: enabling across GCC/GCC High added rollout and governance overhead.

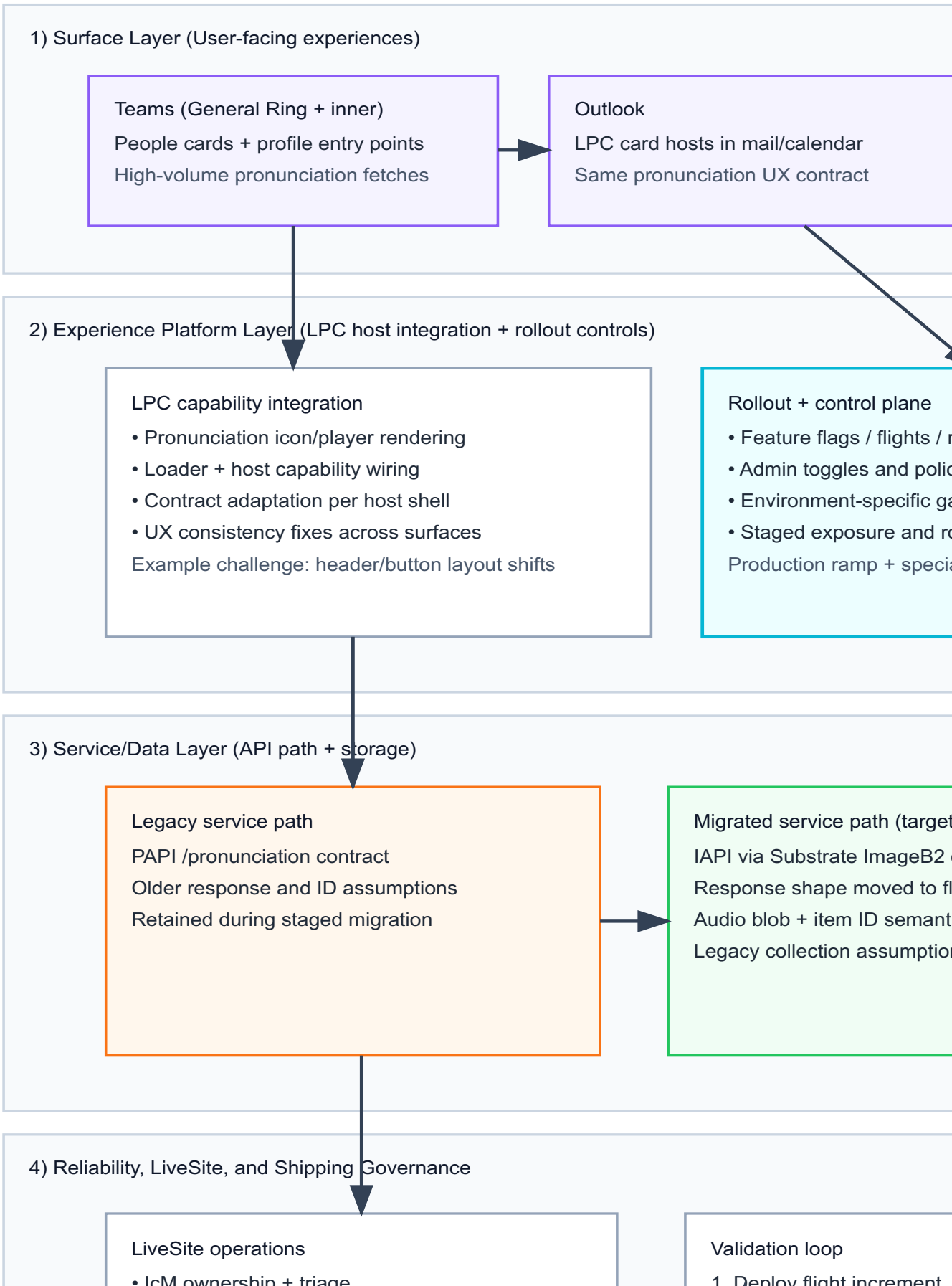
Impact framing

Rollout completion delivered scale of roughly 30M fetch requests/day (~550M/month) on Teams general ring alone, excluding inner rings and special clouds.

Name Pronunciation Detail Break Down

Name Pronunciation Architecture (M365 Multi-Surface)

End-to-end design you helped ship: client experience, API/storage migration, telemetry, and staged rollo



Detailed architecture and delivery narrative

1) Product architecture: what the system does end-to-end

At runtime, the feature is a cross-surface pronunciation retrieval and playback capability embedded in Live People Cards (LPC) and consumed by multiple M365 hosts.

Flow:

- A user opens a person card in a host surface (Teams, Outlook, or another LPC consumer).
- LPC checks host capability + feature gates (flights/admin policy/cloud eligibility).
- LPC invokes the pronunciation capability through the service client path (Loki/client abstraction).
- The request goes through the Name Pronunciation API path (legacy PAPI during transition, target IAPI via Substrate ImageB2).

- Backend resolves the pronunciation item/audio from SDS-accessed item storage in the mailbox-backed model.
- Client receives pronunciation payload (audio + identifiers/metadata contract), renders icon/player, and tracks success/failure telemetry.
- Observability and LiveSite loops monitor reliability and rollout health by ring/surface/cloud.

2) Design choices and why they mattered

A. Shared capability, host-specific shell

You designed this as a single pronunciation capability reused across hosts, rather than separate per-app implementations.

Each host (Teams, Outlook, Vanilla LPC consumers) handled only shell/UI integration, while data behavior stayed centrally controlled.

Why this was a strong choice:

- Faster onboarding to new M365 surfaces.
- Consistent behavior and reliability semantics.
- Lower long-term operational cost (fewer divergent code paths).

B. Ring/flight-first release model

- The feature was shipped via staged rollout controls (rings, flights, admin toggles, environment gates).
- This let you prove reliability incrementally and control blast radius.

Why this mattered:

- Critical for high-scale traffic and sensitive cloud environments.
- Enabled safe rollback/pausing when incidents appeared.

C. Backend migration with compatibility guardrails (PAPI → IAPI)

- You managed migration while live traffic continued.
- Contract differences (payload shape/legacy assumptions) were absorbed progressively.

Why this mattered:

- Avoided hard breaks for host clients.
- Preserved user experience while platform architecture modernized.

3) Implementation architecture: major components

- Surface layer (consumer hosts)
- Teams LPC entry points (largest scale).
- Outlook LPC entry points (mail/calendar contexts).
- Vanilla LPC/Copilot-adjacent consumers.
- Expanded M365 host onboarding model via same capability contract.

Experience platform layer

- LPC loader/integration for pronunciation UI.
- Feature flag checks and ring/capability gating.
- Host-specific UI adaptation and bug fixes (example: icon-induced header/layout shifts).
- Telemetry instrumentation at request and UX interaction points.

Service/data layer

- Legacy PAPI path maintained during transition window.
- Target IAPI path exposed via Substrate ImageB2 endpoint pattern.
- SDS-mediated item/audio retrieval in mailbox-backed storage model.
- Contract evolution: flatter IAPI payload semantics and removal of legacy assumptions.

Reliability/governance layer

- Incident (IcM) triage ownership.
- Reliability scorecards and trend dashboards by ring/surface.
- Promotion criteria for rollout advancement.

- Cloud-specific governance checks for GCC/GCC High.

4) Technologies selected

- Client/platform: LPC integration stack across Teams/Outlook/other consumers.
- Service path: Loki/service client abstraction, PAPI (legacy), IAPI via Substrate ImageB2.
- Storage model: mailbox-backed pronunciation data path, with SDS storage access for item/audio retrieval.
- Operational stack: telemetry pipelines, reliability dashboards, and IcM-based LiveSite processes.
- Release controls: ringed deployment, flight configs, admin toggles, policy and cloud eligibility gates.

5) How the feature was shipped across M365 surfaces

This followed a repeatable surface enablement pipeline:

1. Capability integration in host
    a. Wire pronunciation capability into that host's LPC shell.
    b. Confirm rendering/interaction behavior (icon/player states, fallback behavior).
2. Gate the feature
    a. Add host-specific flight enrollment + admin/policy checks.
    b. Set initial low exposure ring allocation.
3. Observe and stabilize
    a. Track error signatures, missing-pronunciation rates, API failures, and UI regressions.
    b. Run incident-driven fixes and adjust config as needed.
4. Promote rollout
    a. Increase ring allocation once reliability thresholds hold.
    b. Move from inner/controlled audiences to broader general ring.
5. Extend to additional surfaces
    a. Reuse common capability contract for new host surfaces.

    b. Repeat gating + stability criteria per surface.
6. Sovereign cloud enablement
    a. Apply cloud-specific compliance and operational controls.
    b. Promote to GCC/GCC High with stricter governance.

This is why the architecture scaled to broad M365 usage without fragmenting into per-app custom backends.

6) Core challenges encountered and resolved

- Migration contract drift: PAPI vs IAPI payload/ID differences required careful compatibility handling.
- Rollout reliability regressions: missing-pronunciation and failure spikes required staged mitigation.
- Tenant/client configuration issues: environment-specific misconfiguration affected behavior until corrected.
- Cross-host UX consistency: same capability manifested differently per host shell, creating UI edge-case bugs.

- Cloud/governance complexity: sovereign cloud rollout required additional controls and operational discipline.

7) Delivered impact (as captured in your project narrative)

- Feature now serves approximately 30M fetch requests/day and ~550M/month on Teams general ring alone.
- That excludes inner rings and special clouds, so total ecosystem impact is higher.
- Impact came from combined ownership across architecture evolution, migration, rollout, reliability, and cross-surface enablement.

Core challenges: problem, why it was hard, and how you resolved it

1) PAPI → IAPI migration without breaking live traffic

Problem:
The feature had to move from a legacy pronunciation path (PAPI) to IAPI (Substrate/ImageB2) while hosts

were already serving production traffic. The response contract changed (flatter payload, different assumptions around IDs/fields like allowedAudience behavior), and legacy client expectations didn't fully match the new shape.

Why this was hard:

- Multiple hosts (Teams, Outlook, other LPC consumers) were not all on identical client versions.
- A backend contract mismatch in one host can look like "pronunciation missing" to users.
- Migration bugs only appear at real scale and tenant diversity.

Resolution steps taken:

1. **Mapped old vs new contract semantics** field-by-field and identified incompatible assumptions.
2. **Introduced compatibility handling in client/service integration** so hosts could consume new IAPI semantics safely.
3. **Flighted migration by rings** instead of hard-cutting globally.

4. **Tracked migration-specific failure signatures** (especially missing-item/404-like paths).
5. **Fixed config/client path issues** (including tenant-impacting misconfiguration) and validated traffic recovery before wider promotion.

Result:
Migration progressed with controlled blast radius, and monitored error paths (including 404 patterns) were reduced/cleared in key flows.

---

2) Reliability regressions during rollout (missing pronunciation / intermittent failures)

Problem:
As rollout expanded, some users/surfaces saw missing pronunciation or unstable behavior.

Why this was hard:

- Failures could originate from UI integration, feature gating, service path, or tenant/env config.

- The same user-visible symptom (“no pronunciation”) had multiple root causes.

Resolution steps taken:

1. **Centralized incident management (IcM ownership + master tracking)** to prevent fragmented triage.
2. **Clustered incidents by failure signature** (contract, config, service availability, UI rendering).
3. **Used staged allocation changes** (pause/hold/ramp) based on telemetry health gates.
4. **Closed root causes one by one** instead of masking symptoms with broad fallbacks.
5. **Fed each incident learnings back into rollout criteria** so future promotions required stronger reliability evidence.

Result:
Rollout became a controlled reliability program rather than a one-time release event.

---

3) Tenant/environment configuration drift

Problem:
Different environments/tenants had config differences (client pathing, feature enablement, dependency readiness), causing uneven behavior.

Why this was hard:

- Public cloud assumptions don't always hold in every tenant or cloud instance.
- Drift can silently pass in dev/test but fail in specific production slices.

Resolution steps taken:

1. **Built/used explicit environment parity checks** for critical config and dependency assumptions.
2. **Separated code defects from config defects** early in triage.
3. **Corrected misconfigured paths at the right layer** (not patching over in UI).
4. **Added stronger pre-promotion checks** before ring expansion.

Result:
Fewer "works in one environment, fails in another" regressions during later rollout stages.

4) Cross-surface UX consistency issues

Problem:
Adding pronunciation UI into multiple host shells introduced layout and interaction edge cases (for example header/button shift behavior when the icon appears).

Why this was hard:

- LPC capability is shared, but host shells differ in spacing, responsive behavior, and visual constraints.
- Small UI regressions can block broad rollout even when backend is healthy.

Resolution steps taken:

1. **Defined host-specific rendering constraints** while keeping shared capability behavior.
2. **Fixed shell-level layout interactions** instead of forking core pronunciation logic.
3. **Validated UI behavior per host before ring promotion**.

Result:
You preserved reuse of the shared capability while meeting each host's UX quality bar.

---

5) Telemetry and readiness gating at scale

Problem:
You needed confidence to promote rollout at high volume, but raw telemetry alone can be noisy without ring/surface breakdowns and incident correlation.

Why this was hard:

- Scale can hide localized regressions.
- Without clear gates, rollouts become subjective and risky.

Resolution steps taken:

1. **Instrumented by surface/ring/environment** rather than only global success metrics.
2. **Aligned rollout decisions to explicit health thresholds** (not anecdotal confidence).

3. **Connected telemetry with LiveSite/IcM signals** to validate real user impact.

Result:
Promotion decisions became data-governed, supporting safe expansion to very high request volume.

What a cloud/governance deployment is

A **cloud/governance deployment** is a release into environments with stricter compliance, policy, and operational controls (e.g., GCC/GCC High), not just a normal commercial-cloud rollout.

How it differs from a "normal" deployment

Dimension	Normal commercial deployment	Cloud/gover
Compliance controls	Standard enterprise controls	Elevated cont (regulatory/p
Data/tenant boundaries	Broader defaults	Stricter reside boundaries
Service dependency assumptions	Usually uniform and faster-moving	May differ by lag

Dimension	Normal commercial deployment	Cloud/gover
Change velocity	Faster ramp possible	Slower, gated tighter approv
Feature enablement	Flag/ring driven	Flag/ring + policy/admin
Observability and ops	Standard dashboards/process	Often stricter evidence requ

---

Cloud/governance challenges you likely faced on this feature, and how they were resolved

A) Dependency parity gaps across clouds

Challenge: Required backend paths or operational readiness may not match public cloud timing.
Resolution: Treated cloud rollout as a separate promotion track with explicit dependency checks and go/no-go gates per cloud.

B) Policy/admin control alignment

Challenge: Feature might be technically working but blocked by admin/policy state.
Resolution: Added/validated admin-toggle and governance control paths before broader enablement.

C) Config fragmentation risk

Challenge: Cloud-specific configuration can drift from baseline and reintroduce resolved issues.
Resolution: Used environment-specific configuration validation and rollout checklists, then promoted only after parity evidence.

D) Higher burden of proof for stability

Challenge: Governance environments require stronger confidence before expansion.
Resolution: Used telemetry + incident history + staged exposure evidence to justify each promotion step.

E) Slower incident feedback loops

Challenge: Detection and escalation can be slower if instrumentation/paths differ by cloud.
Resolution: Tightened cloud-aware monitoring and incident runbooks, and used controlled ramps to limit impact during validation.

Practical pattern you established (why it worked)

You effectively ran this as a **repeatable release system**:

1. Integrate capability in host shell.
2. Gate with flights/policy controls.
3. Roll out by rings with health thresholds.
4. Triage incidents to root cause (contract/config/UI/service).
5. Fix and revalidate before promotion.
6. Run a distinct governance track for sovereign clouds.
7. Promote to broad production only after stability proof.

That pattern is exactly what enabled both **scale** and **safety** for a feature serving tens of millions of daily requests.